

A Field Guide to Epistemic Failure

Sample — Part One: The Foundation

James Roy Dennis

PortusSophia, LLC

This sample contains the complete first part of the book — four foundation chapters. The full manuscript continues with Part Two: The Catalog (eleven failure modes across three operational strata) and Part Three: The Diagnostic Lens.

Available at leanpub.com

Part One

The Foundation

Chapter 1

What Epistemic Failure Is

and Why Existing Frameworks Cannot See It

You have already experienced epistemic failure.

You may not have called it that. You called it a bad meeting, a failed project, a system that kept breaking in the same place.

Some of those diagnoses were correct. Many were not.

What you were experiencing — underneath the surface event — was something more structural. Something that your existing frameworks had no category for. Something that kept happening not because the people were incompetent or the processes were poorly designed, but because the knowledge that should have been available was not delivered.

This book gives that condition a name. Eleven names, in fact.

The Frameworks You Already Have

You are not without diagnostic tools.

Engineering gave you FMEA — Failure Mode and Effects Analysis. It is rigorous. It assumes the failures are in the physical system.

AI safety gave you MITRE ATLAS. It is growing. It assumes the failures are in the AI component.

Organizational behavior gave you project failure taxonomies and change management frameworks. They assume the failures are in the people or the process.

Each is legitimate. Each is incomplete in the same direction.

None of them ask: *what happens when the knowledge-bearing structure itself breaks down?*

The failure mode taxonomies that exist were built for physical systems, AI components, and organizational processes. They were not built to describe what happens to knowledge as it moves

through systems — as it crosses interfaces, survives transitions, and either remains available or does not.

That is the gap this book fills.

What Epistemic Failure Is

Epistemic failure is not the same as being wrong.

A team makes an incorrect prediction. They test it, update their understanding, and revise. No epistemic failure — the knowledge-bearing structure worked correctly.

Epistemic failure is not ignorance, either.

A team does not yet know something. They recognize the gap, direct attention toward it, and acquire what they need. No epistemic failure.

Epistemic failure is something more specific: it occurs when a knowledge-bearing structure — the mechanism that carries meaning, constraint, and orientation — breaks down in one of a limited number of distinct ways.

The breakdown is often invisible. The system continues operating. Activity accumulates. Audits pass. And yet the knowledge that should be guiding the system is not there — not absent, exactly, but not functioning. Not available at the point of use.

Not the absence of knowledge. The failure of knowledge to function.

Why Existing Frameworks Cannot See It

The frameworks you already have share one assumption: knowledge is a backdrop, not a substrate.

FMEA uses knowledge to analyze systems. It does not analyze knowledge itself.

Organizational frameworks treat knowledge as a resource to be mobilized. Not a structure to be examined.

AI failure taxonomies address outputs and behaviors. Not the knowledge-bearing structure the system depends upon.

In all of them, knowledge is binary — present or absent, available or not. What none captures is how knowledge behaves *under the conditions of its own use*.

Knowledge must cross interfaces intact. It must survive organizational transitions. It must remain available not just in principle — in some record nobody consults — but functionally, at the point of use, in a form that guides coherent action.

When it fails those tests, existing frameworks see the symptoms — redundant work, compliance theater, AI systems consuming tokens rebuilding context given at the start of the session — and diagnose them as management failures or technology gaps.

They do not see the substrate condition generating them.

A Different Kind of Failure

The meeting where the team worked through a problem they had solved eighteen months ago, because the solution did not survive the transition between the team that found it and the team that needed it.

The compliance cycle where every box was checked and the underlying practice had drifted so far from the documentation that the two were no longer in contact.

The AI deployment where something that should have been caught architecturally was discovered empirically, at cost, in production.

The initiative where every participant was committed and the knowledge of why the last three similar efforts had failed was simply not there — not because it had not been generated, but because it had not been carried.

These are not stories about bad people or broken processes. They are stories about knowledge that failed to function.

That breakdown has a name. Eleven names.

How This Book Makes Its Claims

The failure modes in this book carry two distinct claim classes. Some are *structural observations* — conditions architecturally derivable from the nature of knowledge-bearing systems, supported by the classification methodology described in Chapter 3. Others are *illustrations* — coherent and recognizable patterns not yet empirically validated across populations.

Where the distinction matters, it is noted.

This book also applies *observer sovereignty* — the practice of providing structured observations and documented uncertainty rather than resolved conclusions. Where classifications are contested, they are noted as contested. The reader's independent assessment is not only permitted — it is the intended outcome. This vocabulary is offered as a lens. What you see through it belongs to you.

The contribution is diagnostic. Not prescriptive.

Is what I am seeing a failure of effort, process, or technology — or is it a failure of knowledge to function?

If you can hold that question open, you have already begun to see differently.

Here and Now!

Chapter 2

Knowledge as Constraint-Bearing Structure

You already know that knowledge is not just information.

You have seen data that was accurate and useless. Reports that were complete and irrelevant. Documentation that was present and not consulted. Systems that contained everything needed to make a good decision and produced a bad one anyway.

Information is not knowledge. And knowledge is not simply information organized well.

This chapter introduces the concept that underlies everything in this book: knowledge is a constraint-bearing structure. Not a static asset. Not a collection of facts. A structure. One that must survive the conditions of its own use.

What a Structure Does

A structure is not defined by what it contains. It is defined by what it does.

A bridge is not defined by its materials. It is defined by its capacity to bear load across a span. Remove that capacity and what remains is a heap of materials.

Knowledge works the same way. It is defined not by the facts it contains but by its capacity to carry meaning, constraint, and orientation through the conditions a system places upon it. Remove that capacity — through degradation, substitution, or discontinuity — and what remains is data. Information that no longer functions.

You have worked in both kinds of systems. You know the difference by feel before you can name it.

Constraints Are the Structure

A constraint is a boundary condition that restricts the set of permissible system configurations or behaviors — including governance requirements, architectural limitations, and operational dependencies.

When knowledge is carrying its constraints, it does something specific: it narrows the space of what is possible, what is relevant, and what needs to be decided. Data is available. Knowledge narrows.

A compliance team that genuinely knows — functionally knows — that a particular regulatory requirement is structurally determined by their system's architecture does not re-examine it at every audit cycle. The question is closed. Attention goes elsewhere.

A compliance team with the same information stored in a document that nobody consults will re-examine that question every cycle. Not because they are inefficient. Because the constraint is not being carried. The question stays open when it should be closed.

That gap — between knowledge that carries its constraints and information that does not — is where the cost lives.

Survivability Is the Test

A constraint-bearing structure is not evaluated at the moment of its creation. It is evaluated across the conditions of its use.

Knowledge must survive time. A governance determination made in Q1 must remain available and action-guiding in Q3 — not stored in a record nobody can find, but carried in a form practitioners can use.

It must survive transitions. When a team changes, when a system is handed from one group to another, the knowledge that guided prior decisions must travel. If it does not, whoever inherits the system reconstructs it from scratch. That reconstruction has cost.

It must survive scale. Knowledge that functions in a small, stable environment may degrade when exposed to larger teams, faster cycles, or greater distance between those who hold it and those who need it.

It must survive incentive pressure. Knowledge that constrains action is often inconvenient. Institutional pressure toward speed or the appearance of compliance will stress constraint-bearing structures in predictable ways.

A knowledge structure that fails any of these tests has not failed to exist. It has failed to function. That is epistemic failure.

Per angustiam, incrementum — through narrowing, growth. The constraint that generates is structurally distinct from the constraint that suppresses. One produces development through pressure; the other produces

compliance through force. This distinction is the operational expression of the Non-Coercive Emergence axiom that underlies this framework.

The Carrying Function

Carrying is not storage — knowledge stored in a document nobody consults has not been carried. It is not transmission — information received but not actionable has not been carried. Carrying is the active preservation of knowledge's functional capacity across conditions that will test it — meaning, constraint, and orientation intact on the other side of the transition.

When carrying succeeds, prior knowledge remains available without reconstruction. When carrying fails, the system must reconstruct what it already possessed.

The knowledge was already available. The carrying structure failed to deliver it.

What This Means for Your Systems

The knowledge in your systems that is functioning is carrying its constraints. It narrows open questions. It guides coherent action without requiring reconstruction at each transition.

The knowledge that is not functioning may still exist. But it is not being carried. Your system is paying — in redundant work, repeated decisions, coordination overhead — to reconstruct what it already possessed.

Where in your system is knowledge being stored rather than carried?

Here and Now!

Chapter 3

The Structural vs. Empirical Distinction

Not all knowledge is the same kind of thing.

Most systems treat it as though it is. They apply the same retrieval operations, the same verification procedures, the same audit mechanisms, to knowledge that is fundamentally different in how it is established and what it costs to recover.

That uniform treatment is expensive.

Two Kinds of Knowledge

Structurally derivable knowledge is knowledge whose answer can be obtained by analyzing the system's own architecture — its declared conditions, its logical dependencies, its temporal ordering. You do not measure your way to structurally derivable knowledge. You examine the structure.

If your system requires accountability to be assigned before operational authority can be delegated, that sequencing is architecturally determined. The answer is derivable before any measurement occurs.

Empirically discoverable knowledge is knowledge whose answer requires observation, measurement, or interaction with the system under actual conditions. Whether your AI model meets an accuracy threshold is empirically discoverable. You cannot derive it from architecture. You must measure it.

These distinctions are structural. And yet systems routinely confuse them.

Three Formal Criteria

A piece of knowledge is structurally derivable if it satisfies three conditions:

Architectural Necessity — the knowledge follows from the system's design relationships. It does not require observation to establish.

Prior Knowability — the answer is reachable by examining the system's architecture before the system operates.

Measurement Independence — no amount of data collection can validate or refute it. Empirical evidence may illustrate it; measurement cannot change it.

When all three hold, the question is structurally closed. When any fails, the question is empirically open.

What Confusion Costs

When a system treats structurally derivable knowledge as empirically open, it pays the wrong cost for the wrong operation.

A governance team convenes a working group to determine whether an accountability assignment is required — when the requirement follows directly from the framework they operate under, and always has. The work is competent. It is unnecessary.

A compliance process generates a new analysis each cycle of a dependency that has not changed. The finding is the same every time. The cost accumulates.

The reverse confusion is more dangerous. When a system treats empirically open questions as structurally settled, it stops looking at what is actually happening. This is where models drift from the systems they describe.

The SEH Study

As part of the research underlying this book, the author conducted a Structural-Empirical-Hybrid (SEH) classification study of 150 atomic requirements drawn from six major AI governance standards: NIST AI RMF, EU AI Act, GDPR, ISO 42001, FedRAMP, and NIST CSF 2.0.

Results: 87.3% structurally derivable. 7.3% hybrid. 5.3% empirical.

A second finding mattered equally: 44% of requirements exhibited epistemic boundary ambiguity. That ambiguity is not a defect. It is a finding. Many governance requirements occupy the boundary, and systems without a classification mechanism will treat those cases inconsistently.

The knowledge was already available. The system was paying to retrieve it anyway.

Why Systems Conflate the Two

Absence of a classification mechanism. Without a way to ask whether a question is structurally closed or empirically open, every question looks the same.

Institutional incentive toward visible activity. Measurement produces artifacts. Systems under pressure to demonstrate activity favor the operation that leaves a trail.

The carrying failure cascade. When knowledge fails to carry across transitions, a structural answer derived last cycle must be re-derived this cycle. Structurally, it is carrying failure producing unnecessary empirical cost.

Which questions does your system repeatedly work through whose answers are already determined by its architecture?

Here and Now!

Chapter 4

The Common Grammar of Breakdown

You have now seen three things.

Knowledge is a constraint-bearing structure — not a static asset, but a mechanism that must survive the conditions of its own use. Not all knowledge is the same kind of thing. And the systems you already have for diagnosing failure were not built to see the substrate condition that generates these costs.

This chapter assembles those observations into one claim: the eleven failure modes in this book are not eleven unrelated conditions. They are eleven manifestations of one underlying grammar.

The Grammar

Knowledge fails when the relationship between representation and constraint-bearing conditions becomes distorted.

This is the sentence at the center of the book. Every chapter in Part Two is a different way that distortion occurs.

Representation is how knowledge is held within a system — in documents, processes, models, institutional memory, formal structures.

Constraint-bearing conditions are the actual operative conditions the system navigates — the real dependencies, the genuine requirements, the actual environment where decisions must produce their effects.

When the relationship between them is intact, knowledge functions. When that relationship is distorted, knowledge fails to function.

The Morphology

The eleven failure modes are not a list. They are a family — eleven variations on a single pattern, differentiated by where the distortion occurs and how it manifests.

Breakdown Layer	Failure Mode	Stratum
Representation forgotten	Carrying Failure	A — Information Mechanics
Representation lost	Schema Degradation	A — Information Mechanics
Representation applied to wrong class	Epistemic Class Mismatch	A — Information Mechanics
Representation substituted	Proxy Substitution	B — Governance / Operational
Representation mistaken for legitimacy	Legitimacy Inflation	B — Governance / Operational
Representation preserved, meaning dies	Formal Coherence / Semantic Collapse	B — Governance / Operational
Representation hidden	Epistemic Access Suppression	B — Governance / Operational
Representation rigidified	Collapse	C — Ethical / Existential
Representation expanded beyond support	Inflation	C — Ethical / Existential
Representation consumes alternatives	Totalization	C — Ethical / Existential
Representation consumes self	Ego Inflation / Erasure Collapse	C — Ethical / Existential

Why the Morphology Has This Shape

Stratum A failures operate at the level of knowledge structure itself. Stratum B failures operate at the level of institutional process and measurement. Stratum C failures operate at the level of relational coherence and the limits of self-correction.

Cascade directions follow from this layering. Stratum A failures propagate cost into Stratum B. Stratum B failures create the conditions for Stratum C. The cascade is a tendency, not a law.

The Book That Follows

Part Two opens with Stratum A — the failure modes closest to the knowledge structure itself, the most immediately recognizable. Each chapter opens with recognition — what you have already seen without naming it.

Recognition before definition. Pattern before principle.

Part Three will ask what becomes visible once you have the vocabulary. It will not tell you what to do with what you see.

This book's role is to give you the grammar. You have it now.

Which of these distortions — representation forgotten, substituted, hidden, rigidified — have you already encountered in the systems you run?

Here and Now!

This sample contains Part One of A Field Guide to Epistemic Failure.

Part Two — The Catalog presents all eleven failure modes in full: generating conditions, cost signatures, observable patterns, cross-domain examples, and cascade relationships.

Part Three — The Diagnostic Lens asks what becomes visible once you have the vocabulary.

The complete book is available at leanpub.com

James Roy Dennis

PortusSophia, LLC

ORCID: 0009-0000-6689-8763

Per angustiam, incrementum.

© 2026 James Roy Dennis — PortusSophia, LLC